

PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

Tira E-Commerce Project

1.Database Design

1.1Review Table

Attribute	Data Type	Constraint
Review_ID	INT	Primary Key
Customer_ID	INT	Foreign Key
Product_ID	INT	Foreign Key
Review_Text	INT	NOT NULL
Review_Date	INT	NOT NULL

1.2 Rating Table

Attribute	Data Type	Constraint
Rating_ID	INT	Primary Key
Customer_ID	INT	Foreign Key
Product_ID	INT	Foreign Key
Rating	INT	NOT NULL
Rating_Date	DATE	NOT NULL

2.Table Relationships

Customer → Review

A customer can write many reviews.

Product → Review

A product can receive many reviews.

Customer → Rating

A customer can give many ratings.

Product → Rating

A product can receive many ratings.

Review and Rating

Review and Rating tables are connected to the Customer and Product tables through Customer_ID and Product_ID foreign keys.

3. Table Creation

3.1 Create Review Table

```
SQL> CREATE TABLE Review (  
 2     Review_ID NUMBER PRIMARY KEY,  
 3     Customer_ID NUMBER,  
 4     Product_ID NUMBER,  
 5     Review_Text VARCHAR2(500),  
 6     Review_Date DATE NOT NULL,  
 7     CONSTRAINT fk_review_customer  
 8         FOREIGN KEY (Customer_ID)  
 9         REFERENCES Customer(Customer_ID),  
10     CONSTRAINT fk_review_product  
11         FOREIGN KEY (Product_ID)  
12         REFERENCES Product(Product_ID)  
13 );
```

Table created.

3.2 Create Rating Table

```
SQL> CREATE TABLE Rating (  
 2     Rating_ID NUMBER PRIMARY KEY,  
 3     Review_ID NUMBER,  
 4     Rating_Value NUMBER(1) CHECK (Rating_Value BETWEEN 1 AND 5),  
 5     CONSTRAINT fk_rating_review  
 6         FOREIGN KEY (Review_ID)  
 7         REFERENCES Review(Review_ID)  
 8 );
```

Table created.

4.Review Data Insertion

```
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
  3 VALUES (301, 101, 101, 'Good face wash and suitable for daily use', TO_DATE('18-SEP-2026', 'DD-MON-YYYY'));
```

1 row created.

```
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
  3 VALUES (302, 102, 102, 'Moisturizer is smooth and works well', TO_DATE('19-SEP-2026', 'DD-MON-YYYY'));
```

1 row created.

```
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
  3 VALUES (303, 103, 103, 'Shampoo has a pleasant fragrance', TO_DATE('20-SEP-2026', 'DD-MON-YYYY'));
```

1 row created.

```
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
  3 VALUES (304, 104, 104, 'Nice lipstick shade and good quality', TO_DATE('21-SEP-2026', 'DD-MON-YYYY'));
```

1 row created.

```
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
  3 VALUES (305, 105, 105, 'Body lotion feels soft on the skin', TO_DATE('22-SEP-2026', 'DD-MON-YYYY'));
```

1 row created.

5. Rating Data Insertion

```
SQL> INSERT INTO Rating
  2  (Rating_ID, Review_ID, Rating_Value)
  3  VALUES (401, 301, 4);
```

1 row created.

```
SQL> INSERT INTO Rating
  2  (Rating_ID, Review_ID, Rating_Value)
  3  VALUES (402, 302, 5);
```

1 row created.

```
SQL> INSERT INTO Rating
  2  (Rating_ID, Review_ID, Rating_Value)
  3  VALUES (403, 303, 4);
```

1 row created.

```
SQL> INSERT INTO Rating
  2  (Rating_ID, Review_ID, Rating_Value)
  3  VALUES (404, 304, 5);
```

1 row created.

```
SQL> INSERT INTO Rating
  2  (Rating_ID, Review_ID, Rating_Value)
  3  VALUES (405, 305, 3);
```

1 row created.

6.Retrieve Product Review Details

```
SQL> SELECT
  2     r.Review_ID,
  3     r.Customer_ID,
  4     r.Product_ID,
  5     p.Product_Name,
  6     r.Review_Text,
  7     r.Review_Date,
  8     rt.Rating_Value
  9 FROM Review r
 10 JOIN Product p
 11     ON r.Product_ID = p.Product_ID
 12 JOIN Rating rt
 13     ON r.Review_ID = rt.Review_ID
 14 ORDER BY r.Review_Date;
```

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	PRODUCT_NAME	REVIEW_TEXT	REVIEW_DA	RATING_VALUE
301	101	101	Face Wash	Good face wash and suitable for daily use	18-SEP-26	4

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	PRODUCT_NAME	REVIEW_TEXT	REVIEW_DA	RATING_VALUE
302	102	102	Moisturizer	Moisturizer is smooth and works well		

REVIEW_ID CUSTOMER_ID PRODUCT_ID

PRODUCT_NAME

REVIEW_TEXT

REVIEW_DA RATING_VALUE

303 103 103
Shampoo
Shampoo has a pleasant fragrance
20-SEP-26 4

REVIEW_ID CUSTOMER_ID PRODUCT_ID

PRODUCT_NAME

REVIEW_TEXT

REVIEW_DA RATING_VALUE

304 104 104
Lipstick
Nice lipstick shade and good quality
21-SEP-26 5

REVIEW_ID CUSTOMER_ID PRODUCT_ID

PRODUCT_NAME

REVIEW_TEXT

REVIEW_DA RATING_VALUE

305 105 105
Body Lotion
Body lotion feels soft on the skin
22-SEP-26 3

7. Calculate Average Product

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     ROUND(AVG(rt.Rating_Value), 2) AS Average_Rating
  5 FROM Product p
  6 JOIN Review r
  7     ON p.Product_ID = r.Product_ID
  8 JOIN Rating rt
  9     ON r.Review_ID = rt.Review_ID
 10 GROUP BY p.Product_ID, p.Product_Name
 11 ORDER BY Average_Rating DESC;
```

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

104	
Lipstick	5

102	
Moisturizer	5

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

101	
Face Wash	4

103	
Shampoo	

8. Identify Highly Rated Products

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     ROUND(AVG(rt.Rating_Value), 2) AS Average_Rating
  5 FROM Product p
  6 JOIN Review r
  7     ON p.Product_ID = r.Product_ID
  8 JOIN Rating rt
  9     ON r.Review_ID = rt.Review_ID
 10 GROUP BY p.Product_ID, p.Product_Name
 11 HAVING AVG(rt.Rating_Value) >= 4
 12 ORDER BY Average_Rating DESC;
```

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

102	
Moisturizer	
	5

104	
Lipstick	
	5

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

103	
Shampoo	
	4

101	
Face Wash	